22.1 工具系统架构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~900k tokens，输出 ~85k tokens（本章合计）

前几章我们分析了 OpenClaw 的通道层——消息怎么进来、怎么路由到 Agent、怎么把回复发回去。但 Agent 真正的能力不只是聊天，更多来自**工具调用**（Tool Calling）。本章全面解析 OpenClaw 的工具系统，从架构设计到运行时执行，再到权限控制。

22.1.1 TypeBox Schema → JSON Schema 映射

工具的核心数据结构

OpenClaw 里每个工具都是一个 `AgentTool` 对象，定义来自底层的 `pi-agent-core` 库：

```
// @mariozechner/pi-agent-core — AgentTool 类型（简化）
type AgentTool<TParams = unknown, TResult = unknown> = {
  name: string;                        // 工具名称（如 "web_search"）
  label?: string;                      // 人类可读标签
  description: string;                 // 给 LLM 看的工具描述
  parameters: TParams;                 // 参数 Schema
  execute: (params, onUpdate?) => Promise<AgentToolResult<TResult>>;
};
```

其中 `parameters` 字段需要转换成 JSON Schema 格式，作为 LLM API 的 `tools` 参数发送。OpenClaw 用 **TypeBox** 库来定义类型安全的 Schema。

> **衍生解释**：**TypeBox** 是一个 TypeScript JSON Schema 构建器。它用 TypeScript 函数生成 JSON Schema，同时自动推导出对应的 TypeScript 类型。例如 `Type.String()` 生成 `{ "type": "string" }` JSON Schema，且在 TypeScript 中推导为 `string` 类型。这比手写 JSON Schema + 类型定义要安全得多。

TypeBox 定义示例

以 `message` 工具的部分参数为例：

```
// src/agents/tools/message-tool.ts
import { Type } from "@sinclair/typebox";

const schema = Type.Object({
  action: stringEnum(AllMessageActions, {
    description: "消息操作类型",
  }),
  channel: Type.Optional(Type.String()),
  target: Type.Optional(channelTargetSchema({
    description: "Target channel/user id or name.",
  })),
  message: Type.Optional(Type.String()),
  media: Type.Optional(Type.String({
    description: "Media URL or local path.",
  })),
  dryRun: Type.Optional(Type.Boolean()),
});
```

TypeBox 的 `Type.Object()` 会生成如下 JSON Schema：

```
{
  "type": "object",
  "properties": {
    "action": { "type": "string", "enum": ["send", "reply", ...] },
    "channel": { "type": "string" },
    "target": { "type": "string" },
    "message": { "type": "string" },
    "media": { "type": "string", "description": "Media URL or local path." },
    "dryRun": { "type": "boolean" }
  },
  "required": ["action"]
}
```

Schema 归一化

不同 LLM 提供者对 JSON Schema 的支持程度不同。OpenClaw 通过 `normalizeToolParameters()` 和 `cleanToolSchemaForGemini()` 等函数处理兼容性：

```
// src/agents/pi-tools.schema.ts
export function normalizeToolParameters(tool: AnyAgentTool): AnyAgentTool {
  // 确保 parameters 是有效的 JSON Schema 对象
  // 处理 TypeBox 的 $id、$ref 等内部属性
  // 归一化 enum 值等
}
```

Google Gemini 系列模型对 JSON Schema 的支持比较有限，需要额外清理一些不支持的属性。

22.1.2 工具注册表（`src/agents/openclaw-tools.ts`）

工具分层

OpenClaw 的工具分三个层次：

```
层 1: 基础编码工具（pi-coding-agent 提供）
  ├── read      — 读取文件
  ├── write     — 写入文件
  ├── edit      — 编辑文件
  └── apply_patch — 应用 diff 补丁

层 2: OpenClaw 原生工具（openclaw-tools.ts 组装）
  ├── browser       — 浏览器控制
  ├── canvas        — 画布/UI 渲染
  ├── nodes         — 节点/设备管理
  ├── cron          — 定时任务
  ├── message       — 消息发送（跨通道）
  ├── tts           — 文本转语音
  ├── gateway       — Gateway 管理
  ├── agents_list   — Agent 列表查询
  ├── sessions_*    — 会话管理（list/history/send/spawn）
  ├── session_status — 会话状态
  ├── web_search    — 网页搜索
  ├── web_fetch     — 网页获取
  └── image         — 图片处理

层 3: 插件工具（通过 registerTool() 注册）
  ├── memory_search — 记忆搜索
  ├── memory_get    — 记忆检索
  ├── zalouser      — Zalo 操作
  ├── feishu_doc    — 飞书文档
  └── ...           — 其他插件工具
```

工具组装流程

`createOpenClawTools()` 是 OpenClaw 原生工具的工厂函数，根据上下文参数组装工具集：

```typescript
// src/agents/openclaw-tools.ts（简化）
export function createOpenClawTools(options?): AnyAgentTool[] {
  // 条件性创建：某些工具需要特定条件才可用
  const imageTool = options?.agentDir?.trim()
    ? createImageTool({ config, agentDir, modelHasVision })
    : null;
  const webSearchTool = createWebSearchTool({ config, sandboxed });
  const webFetchTool = createWebFetchTool({ config, sandboxed });
  const messageTool = options?.disableMessageTool
    ? null
    : createMessageTool({ /* 通道上下文参数 */ });

  // 组装工具列表
  const tools: AnyAgentTool[] = [
    createBrowserTool({ ... }),
    createCanvasTool(),
    createNodesTool({ ... }),
    createCronTool({ ... }),
    ...(messageTool ? [messageTool] : []),
    createTtsTool({ ... }),
    createGatewayTool({ ... }),
    createAgentsListTool({ ... }),
    createSessionsListTool({ ... }),
    createSessionsHistoryTool({ ... }),
    createSessionsSendTool({ ... }),
    createSessionsSpawnTool({ ... }),
    createSessionStatusTool({ ... }),
    ...(webSearchTool ? [webSearchTool] : []),
    ...(webFetchTool ? [webFetchTool] : []),
    ...(imageTool ? [imageTool] : []),
  ];

  // 合并插件工具
  const pluginTools = resolvePluginTools({
    context: { config, workspaceDir, agentDir, sessionKey, ... },
    existingToolNames: new Set(tools.map(t => t.name)),
    toolAllowlist: options?.pluginToolAllowlist,
  });

  return [...tools, ...pluginTools];
}
```

几个设计要点值得注意：

1. **条件创建**： `imageTool` 需要 `agentDir` ， `webSearchTool` 可能被配置禁用
2. **上下文注入**： `messageTool` 需要知道当前通道、线程等上下文
3. **去重合并**：插件工具通过 `existingToolNames` 避免与内置工具同名

完整工具组装链

从顶层看，工具的组装经过多步处理：

```
// src/agents/pi-tools.ts（简化）
export function createPiTools(options) {
  // 1. 基础编码工具
  const base = codingTools.flatMap((tool) => [...]);

  // 2. OpenClaw 原生工具 + 插件工具
  const openclawTools = createOpenClawTools({ ...options });

  // 3. 通道特定工具
  const channelTools = listChannelAgentTools({ cfg: config });

  // 4. 合并
  const tools = [...base, ...openclawTools, ...channelTools];

  // 5. 应用工具策略过滤
  const filtered = filterToolsByPolicy(tools, effectivePolicy);

  // 6. Schema 归一化（适配不同 LLM）
  return filtered.map(normalizeToolParameters);
}
```

22.1.3 沙箱感知工具过滤

沙箱的概念

OpenClaw 支持在**沙箱模式**下运行 Agent，限制其对文件系统和命令执行的访问。沙箱通过 `sandboxRoot` 参数指定一个根目录，所有文件操作都限制在此目录下：

```
// src/agents/pi-tools.read.ts — 沙箱化的编辑工具
function wrapSandboxPathGuard(tool: AnyAgentTool, root: string): AnyAgentTool {
  return {
    ...tool,
    execute: async (params, onUpdate) => {
      // 检查路径是否在沙箱目录下
      const resolvedPath = path.resolve(root, params.path);
      if (!resolvedPath.startsWith(root)) {
        throw new Error(`Path escapes sandbox: ${params.path}`);
      }
      return tool.execute(params, onUpdate);
    },
  };
}
```

沙箱对工具的影响

不同的沙箱状态会导致不同的工具版本被使用：

工具	非沙箱	沙箱模式
`read`	可读取任意文件	仅限沙箱目录
`write`	可写入任意文件	仅限沙箱目录
`edit`	可编辑任意文件	仅限沙箱目录
`exec`	完整 shell 访问	工作目录限制为沙箱
`web_search`	完整访问	可能被策略禁用
`browser`	完整访问	使用 Bridge URL

子 Agent 工具过滤

当 Agent 派生子 Agent（Subagent）时，子 Agent 的工具集会被额外限制：

```ts
// src/agents/pi-tools.policy.ts
const DEFAULT_SUBAGENT_TOOL_DENY = [
  "sessions_list",    // 会话管理 — 由主 Agent 协调
  "sessions_history",
  "sessions_send",
  "sessions_spawn",
  "gateway",          // 系统管理 — 子 Agent 不应操作
  "agents_list",
  "whatsapp_login",   // 交互式设置 — 不是任务
  "session_status",
  "cron",             // 定时/调度 — 由主 Agent 协调
  "memory_search",    // 记忆 — 应在 spawn 提示中传递
  "memory_get",
];
```

这个设计遵循**最小权限原则**——子 Agent 只需要完成被分配的具体任务，不需要系统级操作权限。

22.1.4 工具辅助函数

参数读取工具

`common.ts` 提供了一组类型安全的参数读取函数，被所有工具共用：

```ts
// src/agents/tools/common.ts

// 读取字符串参数（支持必填/可选/trim 选项）
function readStringParam(params, key, options?) {
  const raw = params[key];
  if (typeof raw !== "string") {
    if (required) throw new Error(`${label} required`);
    return undefined;
  }
  return trim ? raw.trim() : raw;
}

// 读取数值参数（支持整数截断）
function readNumberParam(params, key, options?) {
  // 支持从 string 或 number 类型读取
  // integer: true 时自动截断小数部分
}

// 读取字符串数组参数
function readStringArrayParam(params, key, options?) {
  // 支持 string[] 和 string（自动转为数组）
}

// 读取反应/表情参数
function readReactionParams(params, options?) {
  // 统一解析 emoji + remove 参数
}
```

结果格式化工具

```
// JSON 结果 — 最常用的返回格式
function jsonResult(payload: unknown): AgentToolResult<unknown> {
  return {
    content: [{ type: "text", text: JSON.stringify(payload, null, 2) }],
    details: payload,
  };
}

// 图片结果 — 用于 image、browser 等工具
async function imageResult(params) {
  const content = [
    { type: "text", text: params.extraText ?? `MEDIA:${params.path}` },
    { type: "image", data: params.base64, mimeType: params.mimeType },
  ];
  return await sanitizeToolResultImages(result, params.label);
}
```

操作门控（Action Gate）

`createActionGate()` 提供了一种简洁的方式来控制工具的部分功能：

```
function createActionGate<T>(actions: T | undefined) {
  return (key, defaultValue = true) => {
    const value = actions?.[key];
    return value === undefined ? defaultValue : value !== false;
  };
}

// 使用示例：Discord 的消息操作门控
const gate = createActionGate(cfg.channels?.discord?.actions);
if (gate("send")) {       // 检查 send 操作是否被配置启用
  // 处理 send 操作
}
```

这个模式让配置变得很灵活——管理员可以精确控制每个工具的每个操作是否可用。

本节小结

1. **工具使用 TypeBox 定义参数 Schema**，自动生成 JSON Schema 发送给 LLM，同时保持 TypeScript 类型安全。不同 LLM 的 Schema 兼容性差异由 `normalizeToolParameters()` 处理。
2. **工具分三层组装**：基础编码工具（read/write/edit/exec）→ OpenClaw 原生工具（15+ 种）→ 插件工具（通过 `registerTool()` 注册）。`createOpenClawTools()` 是组装的核心入口。
3. **沙箱模式通过路径守卫限制文件访问**，子 Agent 通过默认 deny 列表进一步限制工具集，遵循最小权限原则。
4. **`common.ts` 提供了统一的参数读取和结果格式化工具集**，包括 `readStringParam()`、`jsonResult()`、`imageResult()` 和 `createActionGate()` 等，被所有工具共享使用。